

Password Manager Project - Complete Explanation Guide

Table of Contents

- 1. [Project Overview](#)
 - 2. [Technologies Used](#)
 - 3. [File Structure](#)
 - 4. [Detailed File Explanations](#)
 - 5. [Program Flow](#)
 - 6. [Security Features](#)
 - 7. [Database Schema](#)
 - 8. [Key Concepts](#)
 - 9. [Viva Questions & Answers](#)
 - 10. [Installation & Usage](#)
-

PROJECT OVERVIEW

This is a **Command-Line Password Manager** built with Python and MySQL. It allows users to:

- Create accounts (signup/login)
- Store passwords securely with encryption
- View, search, and delete saved passwords
- Each user has their own isolated password storage

Key Features

- **Secure Authentication:** User passwords are hashed using SHA-256
 - **Encrypted Storage:** Stored passwords are encrypted using Fernet (AES-128)
 - **User Isolation:** Each user can only access their own passwords
 - **SQL Injection Protection:** Parameterized queries prevent SQL injection attacks
 - **Input Validation:** Comprehensive validation for all user inputs
-

TECHNOLOGIES USED

Technology	Purpose
Python 3.x	Core programming language
MySQL	Relational database for data storage
mysql-connector-python	MySQL database driver for Python
cryptography (Fernet)	Symmetric encryption for password security
hashlib	SHA-256 hashing for user authentication

FILE STRUCTURE

```
password_manager/
|
├── generate_key_file.py      # Creates encryption key (run once)
├── encryption.py            # Encryption/decryption functions
├── config.py                # Configuration settings
├── database.py              # Database setup and connections
├── user_operations.py       # Signup and login functions
├── password_operations.py   # CRUD operations for passwords
├── password_manager.py      # Main program (entry point)
└── key.key                  # Encryption key file (generated)
```

DETAILED FILE EXPLANATIONS

1. generate_key_file.py (Run this FIRST)

Purpose: Creates the encryption key file that will be used to encrypt/decrypt passwords.

```
from encryption import generate_key

key = generate_key()
with open("key.key", "wb") as key_file:
    key_file.write(key)

print("Encryption key generated and saved to key.key")
print("Keep this file safe! Without it, you cannot decrypt passwords.")
```

How it works:

1. Calls `generate_key()` from encryption module
2. Generates a random Fernet encryption key
3. Saves it to a file named `key.key` in binary mode (`wb`)
4. **CRITICAL:** Without this file, you cannot decrypt any passwords

Why it's needed: The same key must be used to both encrypt and decrypt passwords. This file stores that key persistently.

Important Notes:

- Run this file ONLY ONCE during initial setup
- Keep `key.key` file secure and backed up
- If you lose this file, all encrypted passwords become unrecoverable

2. encryption.py (Security Layer)

This file handles all encryption/decryption operations using **Fernet symmetric encryption**.

```
from cryptography.fernet import Fernet
```

Function 1: generate_key()

```
def generate_key():  
    key = Fernet.generate_key()  
    print(key.decode())  
    return key
```

Purpose: Creates a new random encryption key

Details:

- Uses Fernet's built-in key generation
 - Returns key as bytes
 - Prints the key in decoded format for verification
 - Used only once when setting up the project
-

Function 2: load_key()

```
def load_key():  
    import os  
    if not os.path.exists("key.key"):  
        print("ERROR: key.key file not found!")  
        print("Run generate_key_file.py first to create encryption key.")  
        exit()  
  
    with open("key.key", "rb") as key_file:  
        return key_file.read()
```

Purpose: Loads the encryption key from file

How it works:

1. Checks if `key.key` file exists in current directory
2. If missing, displays error message and exits program
3. If found, reads and returns the key in binary format
4. This key is loaded once at program startup

Error Handling: Prevents program from running without encryption key

Function 3: encrypt_password()

```
def encrypt_password(password, key):  
    cipher = Fernet(key)  
    encrypted = cipher.encrypt(password.encode())  
    return encrypted.decode()
```

Purpose: Encrypts a plain text password

Process Flow:

1. **Input:** Plain text password (string), encryption key
2. **Create Cipher:** Initializes Fernet cipher with the key
3. **Encode:** Converts password string to bytes
4. **Encrypt:** Applies Fernet encryption
5. **Decode:** Converts encrypted bytes back to string for storage
6. **Output:** Encrypted password string

Example:

- Input: "MyPassword123"
 - Output: "gAAAAABhXY...encrypted_data...=="
-

Function 4: decrypt_password()

```
def decrypt_password(encrypted_password, key):  
    cipher = Fernet(key)  
    decrypted = cipher.decrypt(encrypted_password.encode())  
    return decrypted.decode()
```

Purpose: Decrypts an encrypted password back to plain text

Process Flow:

1. **Input:** Encrypted password (string), encryption key
2. **Create Cipher:** Initializes Fernet cipher with the same key
3. **Encode:** Converts encrypted string to bytes
4. **Decrypt:** Applies Fernet decryption
5. **Decode:** Converts decrypted bytes to readable string
6. **Output:** Original plain text password

Security Note: Fernet uses:

- AES-128 encryption in CBC mode
 - PKCS7 padding
 - HMAC for authentication
 - Industry-standard and secure
-

3. config.py (Configuration Settings)

Purpose: Central configuration file for the entire project.

```
from encryption import load_key

ENCRYPTION_KEY = load_key()

DB_CONFIG = {
    'host': 'localhost',
    'user': 'root',
    'password': '12345',
    'database': 'password_manager_db'
}
```

Components:

1. ENCRYPTION_KEY

- Loads the encryption key at program startup
- Used throughout the application for encrypting/decrypting passwords
- Loaded once and kept in memory

2. DB_CONFIG Dictionary

- **host**: Database server location (localhost = your computer)
- **user**: MySQL username (default: root)
- **password**: MySQL password (**Note**: Should be secured in production!)
- **database**: Name of the database to use

Why centralize config?

- Makes it easy to change settings in one place
- No need to update multiple files
- Follows the DRY (Don't Repeat Yourself) principle

Production Considerations:

- Use environment variables instead of hardcoded credentials
- Store sensitive data in **.env** files
- Never commit credentials to version control

4. database.py (Database Layer)

This file manages all database connections and setup.

Function 1: get_connection()

```
def get_connection():
    try:
        connection = mysql.connector.connect(
            host = DB_CONFIG['host'],
            user = DB_CONFIG['user'],
            password = DB_CONFIG['password']
        )
        return connection
    except mysql.connector.Error as err:
        print("Error: ", err)
        return None
```

Purpose: Connects to MySQL server WITHOUT specifying a database

When to use:

- Initial setup when the database doesn't exist yet
- Need to create the database itself

Returns:

- Connection object on success
 - None on failure
-

Function 2: get_db_connection()

```
def get_db_connection():
    try:
        connection = mysql.connector.connect(
            host = DB_CONFIG['host'],
            user = DB_CONFIG['user'],
            password = DB_CONFIG['password'],
            database = DB_CONFIG['database']
        )
        return connection
    except mysql.connector.Error as err:
        print("Error: ", err)
        return None
```

Purpose: Connects to MySQL server AND selects the specific database

When to use:

- All normal operations (insert, select, update, delete)
- After the database has been created

Returns:

- Connection object on success

- None on failure

Difference from `get_connection()`:

- Includes `database` parameter
- Used for most operations

Function 3: `setup_database()`

Purpose: Creates the database and tables if they don't exist

```
def setup_database():
    connection = get_connection()

    if connection:
        cursor = connection.cursor()
        cursor.execute("CREATE DATABASE IF NOT EXISTS password_manager_db")
        print("Database 'password_manager_db' ready.")

        cursor.close()
        connection.close()
```

Step 1: Create Database

- Uses `get_connection()` (no database specified)
- Executes `CREATE DATABASE IF NOT EXISTS`
- Safe to run multiple times (won't error if exists)

```
connection = get_db_connection()
if connection:
    cursor = connection.cursor()

    cursor.execute('''
        CREATE TABLE IF NOT EXISTS users(
            id INT AUTO_INCREMENT PRIMARY KEY,
            username VARCHAR(100) UNIQUE NOT NULL,
            password VARCHAR(100) NOT NULL
        )
    ''')
```

Step 2: Create Users Table

Column	Type	Constraints	Purpose
id	INT	PRIMARY KEY, AUTO_INCREMENT	Unique identifier for each user
username	VARCHAR(100)	UNIQUE, NOT NULL	User's login name

Column	Type	Constraints	Purpose
password	VARCHAR(100)	NOT NULL	Hashed password (NOT plain text)

Key Features:

- **AUTO_INCREMENT**: Database automatically assigns IDs (1, 2, 3, ...)
- **UNIQUE**: No two users can have the same username
- **NOT NULL**: All fields must have values

```
cursor.execute('''
    CREATE TABLE IF NOT EXISTS passwords(
        id INT AUTO_INCREMENT PRIMARY KEY,
        user_id INT NOT NULL,
        website VARCHAR(100) NOT NULL,
        web_username VARCHAR(100) NOT NULL,
        web_password TEXT NOT NULL,
        FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE
    )
''')

connection.commit()
print("Table created successfully...")

cursor.close()
connection.close()
```

Step 3: Create Passwords Table

Column	Type	Constraints	Purpose
id	INT	PRIMARY KEY, AUTO_INCREMENT	Unique identifier for each password entry
user_id	INT	FOREIGN KEY, NOT NULL	Links to the user who owns this password
website	VARCHAR(100)	NOT NULL	Name of the website/app
web_username	VARCHAR(100)	NOT NULL	Username for that website
web_password	TEXT	NOT NULL	Encrypted password (TEXT allows longer strings)

Foreign Key Relationship:

- **FOREIGN KEY (user_id) REFERENCES users(id)** - Links to users table
- **ON DELETE CASCADE** - If a user is deleted, all their passwords are automatically deleted

Database Relationship:


```
users (1) ----< (many) passwords
One user can have many password entries
```

5. user_operations.py (User Authentication)

Handles user signup and login functionality.

Function 1: hash_password()

```
def hash_password(password):
    return hashlib.sha256(password.encode()).hexdigest()
```

Purpose: Converts plain text password to secure hash

Process:

1. `password.encode()` - Converts string to bytes
2. `hashlib.sha256()` - Applies SHA-256 hashing algorithm
3. `.hexdigest()` - Converts hash to 64-character hexadecimal string

Example:

- Input: "mypass123"
- Output: "9af15b336e6a9619928537df30b2e6a2376569fcf9d7e773eccede65606529a0"

Why hash passwords?

- One-way function (cannot be reversed)
- Even if database is compromised, attackers can't see actual passwords
- Same password always produces same hash (for verification)
- SHA-256 is cryptographically secure

Function 2: signup()

Purpose: Creates a new user account

```
def signup(username, password):
    hashed_pw = hash_password(password)
```

Step 1: Hash the Password

- Never store plain text passwords
- Hash before any database operations

```
if not username or not password:
    print("\nError: Username and password cannot be empty.")
    return False

if len(username) < 3:
    print("\nError: Username must be at least 3 characters long.")
    return False

if len(password) < 6:
    print("\nError: Password must be at least 6 characters long.")
    return False
```

Step 2: Input Validation

Check	Requirement	Reason
Empty fields	Both must have values	Prevent incomplete accounts
Username length	Minimum 3 characters	Prevent very short usernames
Password length	Minimum 6 characters	Basic security requirement

```
connection = get_db_connection()
if connection is None:
    print("\nError: Database connection failed.")
    return False
cursor = connection.cursor()

try:
    cursor.execute(
        "INSERT INTO users (username, password) VALUES (%s, %s)",
        (username, hashed_pw))
    connection.commit()

    print("\n Signup successful! Welcome ,", username)
    cursor.close()
    connection.close()
    return True
```

Step 3: Insert into Database

- Uses **parameterized query** (%s placeholders)
- Prevents SQL injection attacks
- Stores hashed password (NOT plain text)
- `connection.commit()` saves changes permanently

Security Feature:



```
# UNSAFE - Vulnerable to SQL injection
cursor.execute(f"INSERT INTO users VALUES ('{username}', '{password}')" )

# SAFE - Parameterized query
cursor.execute("INSERT INTO users VALUES (%s, %s)", (username, password))
```

```
except mysql.connector.Error as err:
    print("\nSignup failed. Please try different credentials.")
    cursor.close()
    connection.close()
    return False
```

Step 4: Error Handling

- Catches database errors (e.g., duplicate username)
- Provides user-friendly error message
- Returns False on failure

Common Error: Username already exists (UNIQUE constraint violation)

Function 3: login()

Purpose: Authenticates user and returns their ID

```
def login(username, password):
    hashed_pw = hash_password(password)
```

Step 1: Hash the Entered Password

- Must hash input to compare with stored hash
 - Never compare plain text passwords
-

```
if not username or not password:
    print("\nError: Username and password cannot be empty.")
    return None
```

Step 2: Basic Validation

- Checks for empty fields
 - Returns None on invalid input
-

```
connection = get_db_connection()
cursor = connection.cursor()

try:
    cursor.execute(
        "SELECT id, password FROM users WHERE username = %s",
        (username,))
    result = cursor.fetchone()
```

Step 3: Query Database

- Retrieves user's ID and stored hashed password
- `fetchone()` returns one row or None
- Only fetches data for the specified username

```
cursor.close()
connection.close()

if result and result[1] == hashed_pw:
    print(f"\nLogin successful! Welcome back, {username}!")
    return result[0] # Returns user ID
else:
    print("\nInvalid username or password.")
    return None
```

Step 4: Verify Credentials

- `result[0]` = user ID
- `result[1]` = stored hashed password
- Compares hashed passwords (not plain text)
- Returns user ID on success (needed for all future operations)
- Returns None on failure

Why return user ID?

- Used to filter data (users only see their own passwords)
- Maintains session state
- More efficient than repeatedly querying by username

Security: Never logs or displays actual passwords!

6. password_operations.py (Core Password Management)

Handles all CRUD (Create, Read, Update, Delete) operations for passwords.

Function 1: `add_password()`

Purpose: Stores a new password for a website

```
def add_password(user_id, website, web_username, web_password):  
  
    if not website or not web_username or not web_password:  
        print("\nError: All fields are required.")  
        return False  
  
    if len(website) > 200:  
        print("\nError: Website name is too long (max 200 characters).")  
        return False
```

Step 1: Input Validation

Check	Requirement	Reason
All fields	Cannot be empty	Ensure complete data
Website length	Maximum 200 characters	Database field limit

```
connection = get_db_connection()  
if connection is None:  
    print("\nError: Database connection failed.")  
    return False  
  
cursor = connection.cursor()  
  
try:  
    encrypted_pwd = encrypt_password(web_password, ENCRYPTION_KEY)
```

Step 2: Encrypt the Password

- Uses Fernet encryption with loaded key
- Password is now safe to store in database
- Even if database is compromised, passwords are encrypted

Data Flow:

```
Plain Password → encrypt_password() → Encrypted String → Database
```

```
cursor.execute(  
    "INSERT INTO passwords (user_id, website, web_username, web_password)  
VALUES (%s, %s, %s, %s)",  
    (user_id, website, web_username, encrypted_pwd)
```

```
    )
    connection.commit()
    print("\nPassword added successfully!")
    cursor.close()
    connection.close()
    return True
except Exception as err:
    print("\nError adding password:",err)
    cursor.close()
    connection.close()
    return False
```

Step 3: Insert into Database

- Links to user via `user_id` (foreign key)
- Stores encrypted password (not plain text)
- `connection.commit()` saves permanently
- Returns True on success, False on failure

User Isolation: Each password is linked to specific user through `user_id`

Function 2: `view_passwords()`

Purpose: Displays all stored passwords for the logged-in user

```
def view_passwords(user_id):
    connection = get_db_connection()
    if connection is None:
        print("\nError: Database connection failed.")
        return

    cursor = connection.cursor()

    try:
        cursor.execute(
            "SELECT id, website, web_username, web_password FROM passwords WHERE
user_id = %s",
            (user_id,)
        )
        results = cursor.fetchall()
```

Step 1: Fetch User's Passwords

- `WHERE user_id = %s` ensures users only see their own passwords
- `fetchall()` returns list of all matching rows
- Returns empty list if no passwords stored

Security Feature: User isolation through `WHERE user_id = %s`

```
cursor.close()
connection.close()

if results:
    print("\n" + "="*60)
    print("YOUR SAVED PASSWORDS")
    print("="*60)
    for row in results:
        pwd_id, website, web_username, encrypted_pwd = row
        try:
            decrypted_pwd = decrypt_password(encrypted_pwd,
ENCRYPTION_KEY)
        except Exception:
            decrypted_pwd = "[DECRYPTION FAILED - Key may have changed]"

        print(f"\nID: {pwd_id}")
        print(f"Website: {website}")
        print(f"Username: {web_username}")
        print(f>Password: {decrypted_pwd}")
        print("-"*60)
    else:
        print("\nNo passwords saved yet.")
```

Step 2: Decrypt and Display

- Loops through all results
- Decrypts each password before displaying
- Handles decryption errors gracefully
- Shows formatted output with separators

Data Flow:

Database → Encrypted String → decrypt_password() → Plain Password → Display

Error Handling:

- If decryption fails (e.g., key was changed), shows error message
- Prevents program crash
- User knows which passwords are affected

Function 3: search_password()

Purpose: Finds passwords matching a website name

```
def search_password(user_id, website):
    if not website:
```

```
        print("\nError: Please enter a website name to search.")
        return

    connection = get_db_connection()
    cursor = connection.cursor()

    try:
        cursor.execute(
            "SELECT id, website, web_username, web_password FROM passwords WHERE
            user_id = %s AND website LIKE %s",
            (user_id, f"%{website}%")
        )
        results = cursor.fetchall()
```

Key Feature: LIKE Operator

- LIKE %search_term% enables partial matching
- % is a wildcard that matches any characters
- Case-insensitive search

Examples:

User Searches	Will Find
"face"	Facebook, facebook.com, my-facebook
"gmail"	Gmail, gmail.com, Gmail Personal
"bank"	BankOfAmerica, mybank, onlinebanking

Query Breakdown:

WHERE user_id = %s AND website LIKE %s

↓

User isolation

↓

Partial match search

```
cursor.close()
connection.close()

if results:
    print("\n" + "="*60)
    print(f"SEARCH RESULTS FOR: {website}")
    print("="*60)
    for row in results:
        pwd_id, website, web_username, encrypted_pwd = row
        try:
            decrypted_pwd = decrypt_password(encrypted_pwd,
            ENCRYPTION_KEY)
        except Exception:
```



```
        decrypted_pwd = "[DECRYPTION FAILED]"

        print(f"\nID: {pwd_id}")
        print(f"Website: {website}")
        print(f"Username: {web_username}")
        print(f>Password: {decrypted_pwd}")
        print("-"*60)
    else:
        print(f"\nNo passwords found for '{website}'.")
except mysql.connector.Error as err:
    print(f"\nDatabase error: {err}")
```

Display Results:

- Similar to `view_passwords()`
 - Shows search term in header
 - Decrypts and displays each match
 - User-friendly message if no results found
-

Function 4: `delete_password()`

Purpose: Removes a password entry after confirmation

```
def delete_password(user_id, password_id):
    connection = get_db_connection()
    cursor = connection.cursor()

    try:
        cursor.execute(
            "SELECT website FROM passwords WHERE id = %s AND user_id = %s",
            (password_id, user_id)
        )
        result = cursor.fetchone()

        if not result:
            print("\nPassword not found or you don't have permission to delete
it.")

        cursor.close()
        connection.close()
        return False
```

Step 1: Verify Ownership

- Checks if password exists
- Ensures user owns this password (security)
- `AND user_id = %s` prevents users from deleting others' passwords

Security Check:

```
# WITHOUT user_id check (UNSAFE)
"DELETE FROM passwords WHERE id = %s" # Could delete anyone's password

# WITH user_id check (SAFE)
"DELETE FROM passwords WHERE id = %s AND user_id = %s" # Only user's password
```

```
website = result[0]
confirm = input(f"\nAre you sure you want to delete password for
'{website}'? (yes/no): ")

if confirm.lower() != 'yes':
    print("\nDeletion cancelled.")
    cursor.close()
    connection.close()
    return False
```

Step 2: Confirmation Prompt

- Shows website name for context
- Requires explicit "yes" to proceed
- Any other input cancels deletion
- Prevents accidental deletions

User Experience:

```
Enter password ID to delete: 5
Are you sure you want to delete password for 'Facebook'? (yes/no): yes
Password deleted successfully!
```

```
cursor.execute(
    "DELETE FROM passwords WHERE id = %s AND user_id = %s",
    (password_id, user_id)
)
connection.commit()
print("\nPassword deleted successfully!")
cursor.close()
connection.close()
return True

except mysql.connector.Error as err:
    print(f"\nError deleting password: {err}")
    cursor.close()
    connection.close()
    return False
```

Step 3: Execute Deletion

- Again checks `user_id` for extra security
- Permanently removes record from database
- `commit()` makes deletion permanent
- Returns True/False for success/failure

Why double-check `user_id`?

- Defense in depth
 - Prevents potential security bugs
 - Extra layer of protection
-

7. `password_manager.py` (Main Program / Entry Point)

This is the orchestrator that ties everything together.

Function 1: `main_menu()`

```
def main_menu():
    print("\n" + "="*40)
    print("    PASSWORD MANAGER")
    print("="*40)
    print("1. Signup")
    print("2. Login")
    print("3. Exit")
    print("="*40)
```

Purpose: Displays options for users who haven't logged in yet

Available Actions:

- Signup: Create new account
 - Login: Access existing account
 - Exit: Close the program
-

Function 2: `user_menu()`

```
def user_menu():
    print("\n" + "="*40)
    print("    PASSWORD MANAGER")
    print("="*40)
    print("1. Add Password")
    print("2. View Password")
    print("3. Search Password")
    print("4. Delete Password")
```

```
print("5. Logout")
print("="*40)
```

Purpose: Displays options for logged-in users

Available Actions:

- Add Password: Store new password
 - View Password: See all stored passwords
 - Search Password: Find specific passwords
 - Delete Password: Remove password entry
 - Logout: Return to main menu
-

Function 3: main() - The Heart of the Program

Purpose: Main program loop that handles user interaction

```
def main():
    setup_database()
    current_user_id = None
```

Initialization:

- `setup_database()`: Creates database/tables if they don't exist
- `current_user_id = None`: No one is logged in initially

State Variable:

- `current_user_id` determines which menu to show
 - `None` = not logged in
 - Has value (e.g., 5) = logged in as user with ID 5
-

```
while True:
    if current_user_id is None:
        main_menu()
        try:
            choice = int(input("\nEnter your choice: "))
        except ValueError:
            print("\nInvalid input! Please enter a number.")
            continue
```

Main Loop Structure:

- Runs forever until user exits
- Shows appropriate menu based on login status

- `try-except` handles non-numeric input

Input Validation:

```
# User enters: "abc"
try:
    choice = int("abc") # Raises ValueError
except ValueError:
    print("Invalid input!") # Catches error gracefully
```

When NOT Logged In (`current_user_id` is None):

```
if choice == 1:
    # Signup
    print("\n--- SIGNUP ---")
    username = input("Enter username:")
    password = input("Enter password:")
    signup(username, password)

elif choice == 2:
    # Login
    print("\n--- LOGIN ---")
    username = input("Enter username:")
    password = input("Enter password:")
    user_id = login(username, password)
    if user_id:
        current_user_id = user_id

elif choice == 3:
    # Exit
    print("\nThank you for using Password Manager. Goodbye!")
    break

else:
    print("\nInvalid choice. Please enter 1, 2, or 3.")
```

Option 1: Signup

- Prompts for username and password
- Calls `signup()` function
- Stays at main menu (user must login separately)

Option 2: Login

- Prompts for credentials
- Calls `login()` function
- If successful: stores `user_id` and switches to user menu
- If failed: stays at main menu

Option 3: Exit

- Displays goodbye message
 - `break` exits the infinite loop
 - Program terminates
-

When Logged In (`current_user_id` has a value):

```
else:
    user_menu()
    try:
        choice = int(input("\nEnter your choice: "))
    except ValueError:
        print("\nInvalid input! Please enter a number.")
        continue

    if choice == 1:
        print("\n--- ADD PASSWORD ---")
        website = input("Enter website/app name:")
        web_username = input("Enter website username:")
        web_password = input("Enter password:")
        add_password(current_user_id, website, web_username, web_password)
```

Option 1: Add Password

- Prompts for website, username, and password
 - Calls `add_password()` with current user's ID
 - Password is encrypted before storage
-

```
elif choice == 2:
    view_passwords(current_user_id)
```

Option 2: View Passwords

- Displays all passwords for current user
 - Decrypts passwords for viewing
 - No additional input needed
-

```
elif choice == 3:
    print("\n--- SEARCH PASSWORD ---")
    website = input("Enter website/app name to search: ")
    search_password(current_user_id, website)
```

Option 3: Search Password

- Prompts for website name
- Finds matching passwords using LIKE search
- Shows decrypted results

```
elif choice == 4:
    print("\n--- DELETE PASSWORD ---")
    password_id = input("Enter password ID to delete: ")
    try:
        password_id = int(password_id)
        delete_password(current_user_id, password_id)
    except ValueError:
        print("\nInvalid ID. Please enter a number.")
```

Option 4: Delete Password

- Prompts for password ID (from view/search results)
- Validates ID is a number
- Requires confirmation before deleting

```
elif choice == 5:
    # Logout
    print("\nLogged out successfully!")
    current_user_id = None

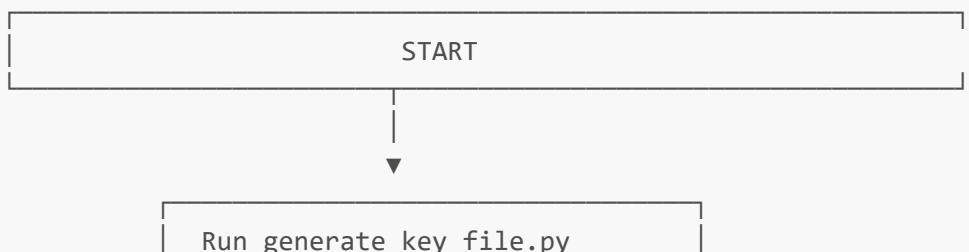
else:
    print("\nInvalid choice. Please try again.")
```

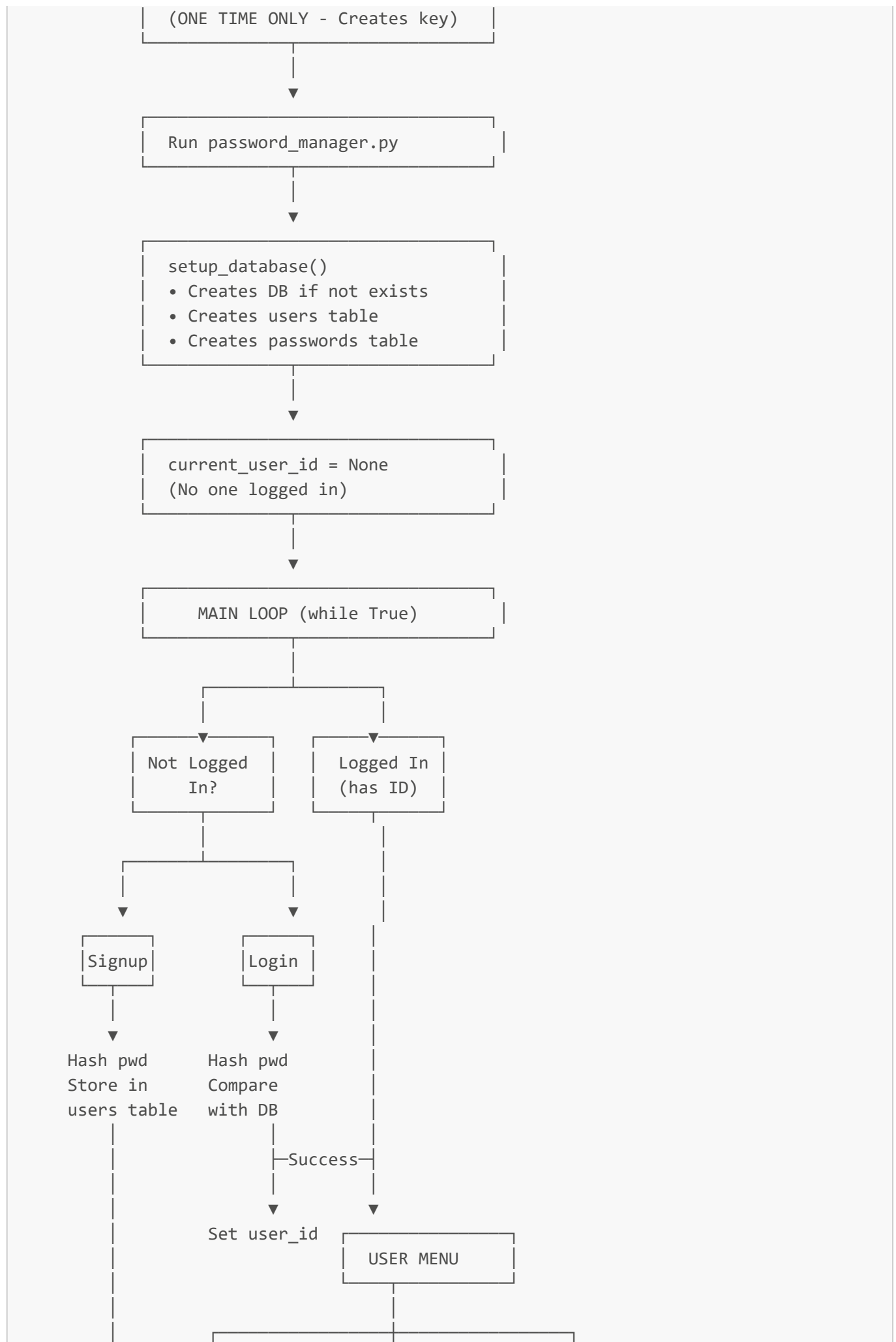
Option 5: Logout

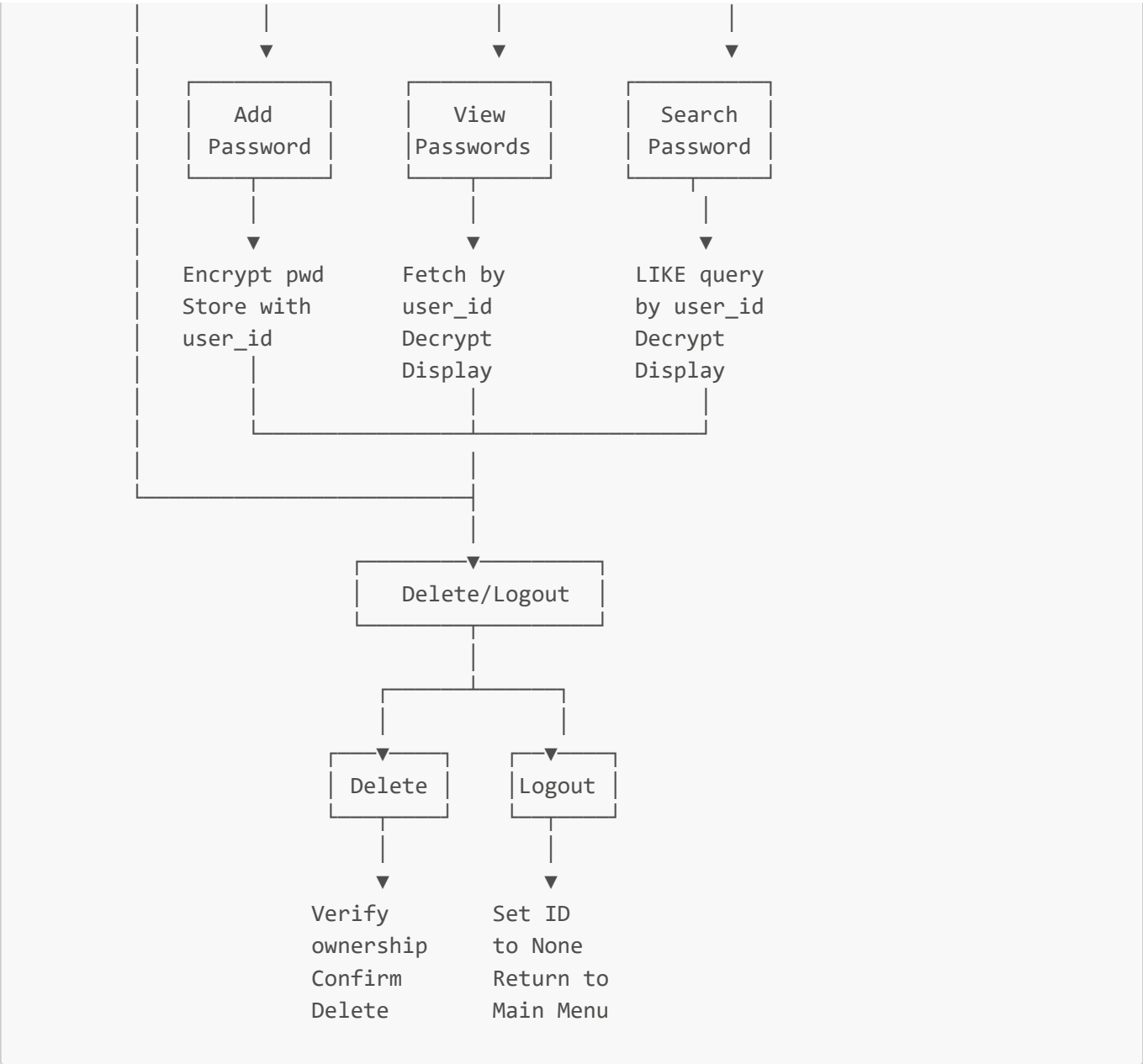
- Resets `current_user_id` to None
- Returns to main menu
- User must login again to access password operations

Important Note: All operations pass `current_user_id` to ensure proper user isolation

PROGRAM FLOW DIAGRAM







SECURITY FEATURES

1. Password Hashing (User Passwords)

What is it?

- User login passwords are hashed using SHA-256
- One-way cryptographic function

How it works:

```
Input: "MyPassword123"
↓
SHA-256 Hash
↓
Output: "9af15b336e6a9619928537df30b2e6a2376569fcf9d7e773eccede65606529a0"
```

Benefits:

- Cannot be reversed to get original password
- Same password always produces same hash (for verification)
- Even database admins can't see actual passwords
- Protects users if database is compromised

Verification Process:

1. User enters password during login
 2. System hashes the input
 3. Compares hashed input with stored hash
 4. If match → login successful
-

2. Password Encryption (Stored Passwords)

What is it?

- Website passwords are encrypted using Fernet (AES-128)
- Two-way encryption (can be decrypted)

How it works:

```
Plain Password: "MyFacebookPass"
    ↓
Fernet Encrypt (with key.key)
    ↓
Encrypted: "gAAAAABhXY...long_encrypted_string...=="
    ↓
Store in Database
    ↓
Decrypt when viewing (with same key.key)
    ↓
Display: "MyFacebookPass"
```

Why encrypt instead of hash?

- Users need to VIEW their stored passwords
- Hashing cannot be reversed
- Encryption can be decrypted with the key

Security:

- Uses AES-128 encryption (industry standard)
 - Requires encryption key to decrypt
 - Even if database is stolen, passwords are encrypted
-

3. SQL Injection Prevention

What is SQL Injection?

- Attacker inserts malicious SQL code through input fields
- Can access/delete/modify database data

Example Attack:

```
# UNSAFE CODE
username = input("Username: ") # User enters: admin' OR '1'='1
query = f"SELECT * FROM users WHERE username = '{username}'"
# Result: SELECT * FROM users WHERE username = 'admin' OR '1'='1'
# This would return ALL users!
```

How we prevent it:

```
# SAFE CODE - Parameterized Query
username = input("Username: ") # User enters: admin' OR '1'='1
cursor.execute("SELECT * FROM users WHERE username = %s", (username,))
# The driver treats input as DATA, not CODE
# SQL injection is prevented!
```

All queries in this project use parameterized queries (safe)

4. User Isolation

What is it?

- Each user can ONLY access their own passwords
- Enforced at database query level

How it works:

```
# Every password operation includes user_id check
cursor.execute(
    "SELECT * FROM passwords WHERE user_id = %s",
    (current_user_id,)
)
```

Example:

- User A (ID: 5) logs in
- User A can only see passwords where `user_id = 5`
- User A cannot access User B's (ID: 7) passwords

Enforced in:

- View passwords
- Search passwords
- Delete passwords
- All operations

5. Input Validation

What is it?

- Checking user input before processing
- Prevents errors and security issues

Validation Rules:

Field	Validation	Reason
Username	Min 3 chars, not empty	Prevent very short usernames
Password	Min 6 chars, not empty	Basic security requirement
Website	Not empty, max 200 chars	Database field constraints
All fields	Cannot be null	Ensure complete data

Example:

```
if not username or not password:
    print("Error: Fields cannot be empty")
    return False

if len(password) < 6:
    print("Error: Password too short")
    return False
```

DATABASE SCHEMA

Complete Database Structure

```
DATABASE: password_manager_db
├── TABLE: users
│   ├── id (INT, PRIMARY KEY, AUTO_INCREMENT)
│   ├── username (VARCHAR(100), UNIQUE, NOT NULL)
│   └── password (VARCHAR(100), NOT NULL)
└── TABLE: passwords
    ├── id (INT, PRIMARY KEY, AUTO_INCREMENT)
    └── user_id (INT, FOREIGN KEY → users.id, NOT NULL)
```

```
|— website (VARCHAR(100), NOT NULL)
|— web_username (VARCHAR(100), NOT NULL)
|— web_password (TEXT, NOT NULL)
```

Users Table

Column	Data Type	Constraints	Description
id	INT	PRIMARY KEY, AUTO_INCREMENT	Unique user identifier
username	VARCHAR(100)	UNIQUE, NOT NULL	User's login name
password	VARCHAR(100)	NOT NULL	SHA-256 hashed password

Sample Data:

```
+-----+-----+-----+-----+
-+
| id | username | password
|
+-----+-----+-----+-----+
-+
| 1 | john      | 5e884898da28047151d0e56f8dc6292773603d0d6aabbdd62a11ef721d1542d8
|
| 2 | alice     | 6b86b273ff34fce19d6b804eff5a3f5747ada4eaa22f1d49c01e52ddb7875b4b
|
+-----+-----+-----+-----+
-+
```

Passwords Table

Column	Data Type	Constraints	Description
id	INT	PRIMARY KEY, AUTO_INCREMENT	Unique password entry ID
user_id	INT	FOREIGN KEY, NOT NULL	Links to users.id
website	VARCHAR(100)	NOT NULL	Website/app name
web_username	VARCHAR(100)	NOT NULL	Username for that website
web_password	TEXT	NOT NULL	Fernet encrypted password

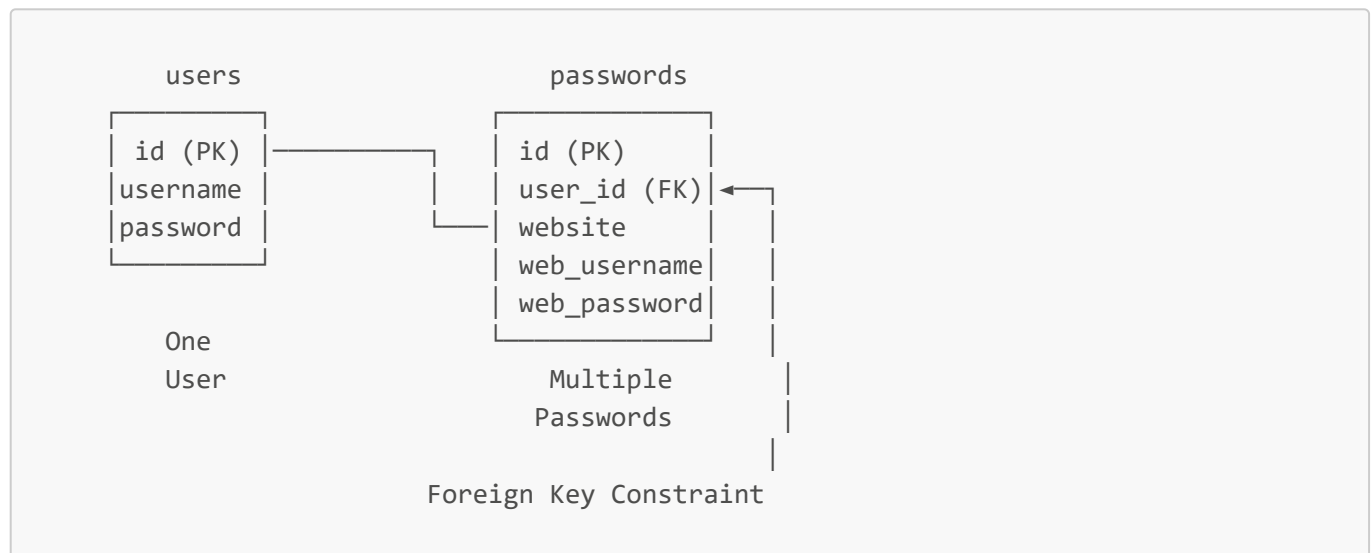
Sample Data:

```
+-----+-----+-----+-----+-----+-----+
+
| id | user_id | website | web_username | web_password
```

1	1	Facebook	john_fb	gAAAAABhXY...encrypted_string...==
2	1	Gmail	john@mail	gAAAAABhXZ...encrypted_string...==
3	2	Twitter	alice_tw	gAAAAABhXa...encrypted_string...==

Database Relationships

One-to-Many Relationship:



Foreign Key Behavior:

- **ON DELETE CASCADE:** If user is deleted, all their passwords are automatically deleted
- Maintains referential integrity
- Prevents orphaned password records

Example:

```
-- User john (id=1) is deleted
DELETE FROM users WHERE id = 1;

-- Automatically deletes:
-- - john's Facebook password (password id=1)
-- - john's Gmail password (password id=2)
-- Alice's passwords (id=3) remain unaffected
```

KEY CONCEPTS FOR VIVA

1. Why separate hashing and encryption?

Hashing (User Passwords):

- **Purpose:** Authentication
- **Direction:** One-way (cannot be reversed)
- **Use Case:** Verify user identity during login
- **Security:** Even admins can't see actual passwords

Encryption (Stored Passwords):

- **Purpose:** Secure storage with retrieval capability
- **Direction:** Two-way (can be decrypted)
- **Use Case:** Show users their website passwords
- **Security:** Requires encryption key to decrypt

Why not encrypt user passwords?

- Don't need to display them back to users
- Hashing is more secure for authentication

Why not hash website passwords?

- Users need to VIEW their stored passwords
- Hashing would make them unrecoverable

2. What happens if key.key is lost?

Consequences:

- All encrypted passwords become **permanently unrecoverable**
- Like losing the key to a safe - safe remains locked forever
- Database still contains data, but it's encrypted gibberish

What users would see:

```
Password: [DECRYPTION FAILED - Key may have changed]
```

Recovery Options:

- **None** - encryption is designed to be unbreakable
- Users would need to:
 - Delete all stored passwords
 - Generate new key.key
 - Re-enter all passwords manually

Best Practice:

- Backup key.key file securely

- Store in multiple secure locations
 - Never commit to version control
-

3. What is AUTO_INCREMENT?

Definition:

- Database automatically assigns unique, incrementing IDs
- Starts at 1, increases by 1 for each new record

How it works:

```
-- First user signup
INSERT INTO users (username, password) VALUES ('john', 'hash123');
-- Database assigns id = 1

-- Second user signup
INSERT INTO users (username, password) VALUES ('alice', 'hash456');
-- Database assigns id = 2

-- Third user signup
INSERT INTO users (username, password) VALUES ('bob', 'hash789');
-- Database assigns id = 3
```

Benefits:

- No need to manually track IDs
- Guaranteed unique IDs
- Simplifies record management
- Primary key for relationships

Used in:

- users.id
 - passwords.id
-

4. What is CASCADE DELETE?

Definition:

- Automatic deletion of related records when parent record is deleted
- Maintains referential integrity

SQL Syntax:

```
FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE
```


How it works:

Before Deletion:



After Deleting User john (id=1):



Why it's important:

- Prevents orphaned records (passwords without users)
- Automatic cleanup
- Data consistency

5. Why check `current_user_id` is `None`?

Purpose:

- Determines application state
- Controls which menu to display

State Management:

```
current_user_id = None # Not logged in
↓
Show main_menu() → Signup/Login/Exit
```

```
current_user_id = 5 # Logged in as user with ID 5
↓
Show user_menu() → Add/View/Search/Delete/Logout
```

Why use None?

- Clear indication of "no user"
- Python convention for "no value"
- Easy to check: `if current_user_id is None`

Alternative approaches (not used):

```
current_user_id = 0      # Could confuse with valid ID
current_user_id = -1     # Not intuitive
current_user_id = False  # Type inconsistency
current_user_id = None   # ✓ Clear and Pythonic
```

POSSIBLE VIVA QUESTIONS & ANSWERS

General Questions

Q1: What is the purpose of this project? **A:** This is a command-line password manager that securely stores website passwords for users. It provides authentication, encrypted storage, and allows users to manage their passwords through various operations like add, view, search, and delete.

Q2: Why did you choose MySQL? **A:** MySQL is:

- Reliable and mature database system
- Widely used in industry
- Supports relational data (perfect for user-password relationships)
- Has excellent Python support via mysql-connector
- Free and open-source
- Handles concurrent users well

Q3: Is this project production-ready? **A:** No. For production deployment, we would need:

- Environment variables for database credentials (not hardcoded)
- Salt added to password hashing (prevents rainbow table attacks)
- HTTPS for secure data transmission
- Two-factor authentication (2FA)
- Password strength requirements and validation
- Rate limiting to prevent brute force attacks
- Logging and monitoring
- Regular security audits
- Backup and disaster recovery systems

Security Questions

Q4: What is Fernet encryption? A: Fernet is a symmetric encryption specification that:

- Uses AES-128-CBC for encryption
- Includes HMAC (Hash-based Message Authentication Code) for authentication
- Ensures both confidentiality (data cannot be read) and integrity (data cannot be modified)
- Part of Python's cryptography library
- Industry-standard and secure

Q5: Can two users have passwords for the same website? A: Yes, absolutely! Each password entry is linked to a specific user via `user_id` foreign key. Multiple users can store passwords for "Facebook" or any website without conflict. The database design ensures complete user isolation.

Example:

- User john (id=1) stores Facebook password
- User alice (id=2) stores Facebook password
- Both entries are separate and isolated

Q6: What prevents SQL injection in your code? A: We use **parameterized queries** throughout the application. Instead of concatenating user input into SQL strings, we use placeholders (`%s`) and pass values as tuple parameters. The MySQL connector handles proper escaping and treats input as data, not executable code.

Example:

```
# UNSAFE
query = f"SELECT * FROM users WHERE username = '{username}'"

# SAFE (what we use)
cursor.execute("SELECT * FROM users WHERE username = %s", (username,))
```

Q7: What happens if someone guesses the encryption key? A: The Fernet encryption key is 32 bytes (256 bits) randomly generated. There are 2^{256} possible keys, making brute force practically impossible. Even with supercomputers, it would take billions of years to try all combinations. The key must be kept secure - anyone with the key can decrypt all passwords.

Technical Questions

Q8: Explain the difference between `get_connection()` and `get_db_connection()`. A:

- **`get_connection()`:** Connects to MySQL server WITHOUT selecting a database. Used for initial setup to create the database itself.

- **get_db_connection():** Connects to MySQL server AND selects the specific database. Used for all normal operations after database exists.
-

Q9: Why use TEXT datatype for web_password instead of VARCHAR? A: Encrypted passwords can be quite long (Fernet produces base64-encoded strings). TEXT datatype allows up to 65,535 characters, while VARCHAR has a 255-character limit in older MySQL versions. Using TEXT ensures we never truncate encrypted passwords.

Q10: What is the purpose of connection.commit()? A: Database transactions are not automatically saved. Changes remain in a temporary state until `commit()` is called. This ensures:

- Data integrity (all changes or none)
 - Rollback capability if errors occur
 - ACID properties of databases Without `commit()`, changes would be lost when connection closes.
-

Q11: How does the LIKE operator work in search? A: LIKE enables pattern matching in SQL:

- `%` is a wildcard matching any characters
- `%facebook%` matches: "facebook", "Facebook.com", "my-facebook-account"
- Case-insensitive by default in MySQL
- Very useful for flexible searching

Example:

```
WHERE website LIKE '%gmail%'
Finds: Gmail, gmail.com, Gmail Personal, mygmail
```

Database Questions

Q12: Explain the foreign key relationship in your database. A: The `passwords` table has a foreign key `user_id` that references `users.id`. This creates a parent-child relationship where:

- One user (parent) can have many passwords (children)
 - Each password must belong to exactly one user
 - `ON DELETE CASCADE` ensures when a user is deleted, all their passwords are automatically deleted
 - Maintains referential integrity
-

Q13: What is referential integrity? A: Referential integrity ensures that relationships between tables remain consistent. In our project:

- Every `user_id` in `passwords` table must exist in `users` table
 - Cannot create a password for a non-existent user
 - Foreign key constraints enforce this automatically
 - Prevents orphaned records and data inconsistencies
-

Q14: Why is username UNIQUE in the users table? A: The UNIQUE constraint ensures no two users can have the same username. This is essential because:

- Usernames are used for login identification
 - Prevents confusion and conflicts
 - Enforces business logic at database level
 - Attempting to insert duplicate username raises an error
-

Design Questions

Q15: Why separate files instead of one large file? A: This follows the **separation of concerns** principle:

- **Modularity:** Each file has a specific responsibility
 - **Maintainability:** Easy to find and fix bugs
 - **Reusability:** Functions can be imported elsewhere
 - **Readability:** Smaller, focused files are easier to understand
 - **Collaboration:** Multiple developers can work on different files
-

Q16: What improvements would you make to this project? A:

1. **GUI:** Add graphical interface using Tkinter or PyQt
 2. **Password Generator:** Built-in strong password generator
 3. **Master Password:** Single password to access the manager
 4. **Password Strength Meter:** Visual indicator of password strength
 5. **Export/Import:** Backup passwords to encrypted files
 6. **Password History:** Track password changes over time
 7. **Expiry Reminders:** Notify users to change old passwords
 8. **Multi-factor Authentication:** Add extra security layer
 9. **Cloud Sync:** Synchronize across devices
 10. **Browser Extension:** Auto-fill passwords in browsers
-

Q17: How would you handle multiple users accessing the database simultaneously? A: MySQL handles concurrent access through:

- **Transactions:** Ensure atomic operations
 - **Locking:** Prevents conflicting modifications
 - **Isolation:** Each user sees consistent data Our current implementation supports multiple users since each operation is isolated by `user_id` and MySQL handles concurrency automatically.
-

Q18: What testing would you perform on this project? A:

1. **Unit Testing:** Test each function individually
2. **Integration Testing:** Test file interactions
3. **Security Testing:** SQL injection, encryption validation
4. **User Testing:** Real users try the interface
5. **Performance Testing:** Test with large datasets

6. **Error Handling:** Test invalid inputs and edge cases
 7. **Database Testing:** Test constraints and relationships
-

INSTALLATION & USAGE

Prerequisites

```
# Install Python 3.x (if not already installed)
# Download from python.org

# Install MySQL Server
# Download from mysql.com

# Install required Python packages
pip install mysql-connector-python
pip install cryptography
```

Setup Instructions

Step 1: Start MySQL Server

```
# Windows: Start MySQL from Services
# Mac: brew services start mysql
# Linux: sudo systemctl start mysql
```

Step 2: Configure Database Credentials Edit `config.py` with your MySQL credentials:

```
DB_CONFIG = {
    'host': 'localhost',
    'user': 'your_mysql_username',
    'password': 'your_mysql_password',
    'database': 'password_manager_db'
}
```

Step 3: Generate Encryption Key (ONE TIME ONLY)

```
python generate_key_file.py
```

Output:

```
<encryption_key_displayed>
Encryption key generated and saved to key.key
Keep this file safe! Without it, you cannot decrypt passwords.
```

IMPORTANT: Backup this `key.key` file securely!

Step 4: Run the Password Manager

```
python password_manager.py
```

Usage Example

```
=====
      PASSWORD MANAGER
=====
1. Signup
2. Login
3. Exit
=====

Enter your choice: 1

--- SIGNUP ---
Enter username: john
Enter password: mypass123

Signup successful! Welcome, john

=====
      PASSWORD MANAGER
=====
1. Signup
2. Login
3. Exit
=====

Enter your choice: 2

--- LOGIN ---
Enter username: john
Enter password: mypass123

Login successful! Welcome back, john!

=====
      PASSWORD MANAGER
```

```
=====
1. Add Password
2. View Password
3. Search Password
4. Delete Password
5. Logout
=====

Enter your choice: 1

--- ADD PASSWORD ---
Enter website/app name: Facebook
Enter website username: john.doe@email.com
Enter password: fb_secure_pass_123

Password added successfully!

Enter your choice: 2

=====
YOUR SAVED PASSWORDS
=====

ID: 1
Website: Facebook
Username: john.doe@email.com
Password: fb_secure_pass_123
-----
```

PROJECT STRUCTURE SUMMARY

```
password_manager/
|
├── generate_key_file.py    # ONE-TIME: Creates encryption key
│   └── Creates: key.key
|
├── encryption.py          # Encryption/decryption utilities
│   ├── generate_key()
│   ├── load_key()
│   ├── encrypt_password()
│   └── decrypt_password()
|
├── config.py              # Configuration settings
│   ├── ENCRYPTION_KEY
│   └── DB_CONFIG
|
├── database.py            # Database management
│   ├── get_connection()
│   ├── get_db_connection()
│   └── setup_database()
```



```
├── user_operations.py      # User authentication
│   ├── hash_password()
│   ├── signup()
│   └── login()
├── password_operations.py # Password CRUD operations
│   ├── add_password()
│   ├── view_passwords()
│   ├── search_password()
│   └── delete_password()
└── password_manager.py    # Main program
    ├── main_menu()
    ├── user_menu()
    └── main()
```

DATA FLOW SUMMARY

User Signup Flow

User Input → hash_password() → Store in users table

User Login Flow

User Input → hash_password() → Compare with DB → Return user_id

Add Password Flow

Plain Password → encrypt_password() → Store with user_id → Database

View Password Flow

Database → Fetch by user_id → decrypt_password() → Display

Search Password Flow

Search Term → LIKE query → Filter by user_id → Decrypt → Display

Delete Password Flow

```
Password ID → Verify ownership → Confirmation → Delete from DB
```

PROJECT STRENGTHS

✓ **Modular Design:** Separate files for different functionalities ✓ **Security:** Hashing for authentication, encryption for storage ✓ **SQL Injection Protection:** Parameterized queries throughout ✓ **User Isolation:** Users can only access their own data ✓ **Error Handling:** Comprehensive try-except blocks ✓ **Input Validation:** Checks for valid data before processing ✓ **Database Constraints:** UNIQUE, NOT NULL, FOREIGN KEY ✓ **Clear Code Structure:** Well-organized and commented ✓ **User-Friendly:** Clear prompts and error messages ✓ **Scalable:** Can easily add more features

AREAS FOR IMPROVEMENT

✗ **No Master Password:** Application doesn't require password to start ✗ **Hardcoded Credentials:** DB password in config.py ✗ **No Password Strength Requirements:** Accepts weak passwords ✗ **No Password Generator:** Users must create own passwords ✗ **Command-Line Interface:** Could benefit from GUI ✗ **No Backup/Export:** Cannot export passwords ✗ **No Password History:** Cannot track password changes ✗ **No Multi-Factor Auth:** Single authentication factor ✗ **No Session Timeout:** Users remain logged in indefinitely ✗ **No Audit Logging:** No record of who accessed what

CONCLUSION

This Password Manager project demonstrates fundamental concepts in:

- Database design and management
- User authentication and authorization
- Cryptography (hashing and encryption)
- Security best practices
- Python programming
- SQL and relational databases

While suitable for learning and demonstration purposes, additional security features would be necessary for production deployment.

QUICK REFERENCE

Important Commands

```
# Generate encryption key (run once)
python generate_key_file.py
```

```
# Run the application
python password_manager.py

# Install dependencies
pip install mysql-connector-python cryptography
```

Key Files

- `key.key` - Encryption key (KEEP SECURE!)
- `config.py` - Database credentials
- `password_manager.py` - Main entry point

Database

- Name: `password_manager_db`
- Tables: `users`, `passwords`
- Relationship: One-to-Many (user to passwords)

Security Features

- SHA-256 hashing for user passwords
- Fernet (AES-128) encryption for stored passwords
- Parameterized SQL queries
- User isolation via `user_id`
- Input validation

End of Document

This guide covers all aspects of the Password Manager project. Study each section carefully and practice explaining the concepts in your own words for the viva examination. Good luck!